Diagonal Hyperreflection over Dynamic-Kind System Fω

A Computable and Formally Dominating Contender 14 for BusyLean

Research memorandum for the BusyLean project

October 12, 2026

Abstract

We propose and formally implement a new BusyLean number, `contender_14`, that strictly exceeds the repository's dynamic-kind System Fω Contender 13. The construction has two ingredients. First, it upgrades the unary oracle of Contender 13 to a countable oracle family and packages dynamic-kind reflection as a progressive transformer. Second, it introduces a *diagonal hyperreflection hierarchy*: rank zero is the repository's autonomous transformer lift; at successor rank, the preceding rank is used both as the local lifting operation along a well-founded schedule and as a pre-amplification of the base transformer. The rank and the schedule are themselves indexed by Contender 13's entire family, rather than by a fixed small numeral.

The formal development proves more than the minimum contest comparison. Every diagonal rank is progressive; every rank is strictly inflationary over its base; consecutive ranks are strictly increasing at every visible coordinate; every dynamic-kind program within the current syntax budget is below the resulting family; and finally

$$\texttt{contender_13} < \texttt{contender_14}.$$

The comparison avoids the invalid assumption that reflection is monotone in its oracle. Every inequality is instead witnessed by an explicit occurrence of a seed or child transformer. The closed natural-number definition is structurally recursive and computable in principle. Lean 4.34.0 validation succeeds, including the repository's time budgets, an axiom audit, a forbidden-construct scan, and a transitive-closure audit showing that no earlier contender constant occurs in the definition.

Contents

1 Executive summary **4**

2 Repository baseline and design constraints **4**
2.1 The accepted ladder through Contender 13 4
2.2 Contest constraints 5

3 Families, transformers, and progressivity **6**

4 A generic stage for arbitrary transformer lifts **6**

5 The diagonal hyperreflection hierarchy **8**
5.1 Formal rank theorems 8

6 Dynamic-kind reflection over a countable family **9**

7 Definition of Contender 14 **9**
7.1 Previous construction as a family . . . 9
7.2 Input-dependent schedule and rank . . . 10
7.3 Final family and number . . . 10

8 Formal comparison and domination results **10**
8.1 The previous family is below the new family . . . 10
8.2 Raw dynamic-kind reflection is below the hyperreflection layer . . . 11
8.3 Every enumerated dynamic-kind program is below F_{14} . . . 11
8.4 The required contender comparison . . . 11

9 Why the gain is structurally significant **11**
9.1 A previous-family-sized strict rank chain . . . 12
9.2 Two uses of the preceding rank at each successor . . . 12
9.3 Countable-family access at the strongest language level . . . 12
9.4 Schedule growth is also previous-family indexed . . . 12

10 Computability, constructivity, and axiom profile **12**
10.1 Computable in principle . . . 12
10.2 Axiom audit . . . 13
10.3 Closure audit . . . 13
10.4 Forbidden-construct scan . . . 13

11 Machine validation **13**
11.1 Full clean dependency pass . . . 13
11.2 Budget-faithful cached pass . . . 14

12 Proof-theoretic interpretation: what is and is not claimed **14**

13 Threats to validity and adversarial review **15**
13.1 Could the definition hide the previous contender? . . . 15
13.2 Does the comparison rely on oracle monotonicity? . . . 15
13.3 Is `Autonomous.hyper` semantically overclaimed? . . . 15
13.4 Does the huge rank cause nontermination of the definition checker? . . . 15
13.5 Is the family reflector merely adding the seed by a maximum? . . . 15
13.6 Could the proof be vacuous because the term enumeration is empty? . . . 15
13.7 Are allowed axioms used computationally? . . . 15

14 Integration plan **15**

15 Further research questions **16**
15.1 Semantic calibration of diagonal hyperreflection . . . 16
15.2 Transfinite rather than natural rank syntax . . . 16
15.3 Internalizing family access in the object language . . . 16
15.4 A predicative ladder of dynamic kinds . . . 17
15.5 Large elimination with realizability . . . 17
15.6 Compression and kernel performance . . . 17

15.7 Formal contest dominance beyond the immediate predecessor 17
15.8 Independent mechanization 17

16 Conclusion **17**

A Formal theorem inventory **18**

B Artifact map **18**

1 Executive summary

The current repository already contains a substantial ladder of formally verified large numbers. Contender 12 applies operator-level reflection over kind-polymorphic System Fω along an explicit autonomous tree schedule. Contender 13 then makes a qualitative language jump: it reflects a dynamic-kind extension of System Fω with a predicative universe of operators, type-level packing and unpacking, and two levels of kind quantification. Contender 13 is therefore the correct baseline; a proposal aimed only at Contender 11 or 12 would already be obsolete.

The new proposal exploits a gap between the two latest constructions. Contender 13 has the strongest object language in the repository, but applies it only through one ordinary unary reflection step. Contender 12 has the strongest generic amplifier, but its base language is weaker. The first obvious idea is to put Contender 13's reflector into Contender 12's operator stage. That is valid and already yields a larger number, but it leaves the amplification rank fixed. The present proposal goes further: it iterates the *lifting mechanism itself* through a hierarchy whose rank is larger than Contender 13's value at the same input.

For input n, define

$$\begin{aligned} \mathrm{family}_{13}(n) &:= F_{13}(F_{13}(n)), \\ r_{14}(n) &:= \mathrm{family}_{13}(n) + 1, \\ S_{14}(n) &:= \mathrm{realize}\Big(\mathrm{hyper}_{\mathrm{family}_{13}(n)+42}(\mathrm{encode}(\mathrm{tree}_{12}))\Big). \end{aligned}$$

The final family is

$$F_{14}(n) := \mathsf{D}^{S_{14}(n)}_{r_{14}(n)}(R_{\mathsf{Fd}})(\lambda i.\, \mathrm{family}_{13})(n, n),$$

where R_{Fd} is countable-family reflection for dynamic-kind System Fω and D^S_k is the new diagonal hierarchy of transformer lifts. Finally,

$$\mathrm{contender}_{14} := F_{14}(F_{14}(42)).$$

The formal margin is substantial in four independent senses:

(i) $F_{14}(n)$ exceeds the entire previous family at every input n;

(ii) it exceeds the raw dynamic-kind family-reflection layer at every input;

(iii) it exceeds every enumerated dynamic-kind program in the current syntax budget;

(iv) it sits at the top of a pointwise strict chain of transformer ranks of length $\mathrm{family}_{13}(n) + 1$.

Only the first and a growth-in-input lemma are needed for the contest theorem. The other results document why the construction is not merely a one-unit wrapper around the old number.

2 Repository baseline and design constraints

2.1 The accepted ladder through Contender 13

The relevant recent stages are summarized in Table 1. This table is a code-level description, not a claim of exact proof-theoretic ordinals.

The word "reflection" here is operational: a finite syntax enumeration is type checked, normalized by a total Lean function justified by a strong-normalization theorem, decoded as a natural number,

Stage	Main mechanism	Salient gain
11	Forest reflection over kind-polymorphic Fω	Applies the W-language tree amplifier to a stronger language.
12	Operator-level reflection plus an autonomous schedule	Lifts from families to transformers and iterates the lift along a Veblen-style tree syntax.
13	Dynamic-kind System Fω reflection	Adds a predicative universe of operators and two-level kind quantification, but only one ordinary reflection layer.
14	Input-indexed diagonal hyperreflection over dynamic-kind family reflection	Iterates the transformer-lifting mechanism at a previous-family-sized rank and schedule.

Table 1: The local design trajectory.

and bounded by a new natural number. It is related in spirit to proof-theoretic reflection and ordinal iteration, but the formal object in BusyLean is a concrete computable bound, not an abstract consistency principle.

2.2 Contest constraints

The implementation was designed against the repository's explicit rules:

1. The entry must be a closed, constructive `Nat`. It may use no Busy Beaver or undecidable oracle and no nontermination, foreign primitive, or opaque stand-in.
2. The comparison with the preceding entry must be a Lean theorem.
3. Only the already permitted axioms `propext` and `Quot.sound` may occur in the transitive dependency closure.
4. The new definition and proof layers must fit the stated 15-second and 60-second budgets after earlier accepted modules are precompiled.
5. The definition may reuse the preceding construction as a family, but must not directly insert the constant `contender_13` and add a trivial amount.
6. No monotonicity of reflection in its oracle may be assumed. Such monotonicity is generally false because reflected programs can inspect an oracle result and branch on it.

The sixth point is mathematically important. Many apparently natural hierarchy proofs silently require

$$f \le g \quad \Longrightarrow \quad \mathrm{Reflect}(f, n) \le \mathrm{Reflect}(g, n),$$

but an object-language program may return a huge value when the oracle is even and a small value when it is odd. Increasing the oracle pointwise can therefore lower a program value. The repository's sound comparison style is instead *occurrence based*: arrange for the smaller quantity to appear literally among the values bounded by the larger reflection step. The present development follows that discipline throughout.

3 Families, transformers, and progressivity

A BusyLean oracle family is a function

$$\mathcal{F} := \mathbb{N} \to \mathbb{N} \to \mathbb{N}.$$

The first argument names an oracle coordinate; the second is the ordinary numeric input. A transformer is

$$\mathcal{T} := \mathcal{F} \to \mathcal{F}.$$

Thus a transformer accepts a countable family and returns a countable family.

The key order property is *progressivity*.
Definition 3.1 (Progressive transformer)**.** A transformer T is progressive when

$$\mathsf{Prog}(T) :\Longleftrightarrow \forall s \in \mathcal{F}\ \forall i, o, n \in \mathbb{N}, \quad i \le n \Longrightarrow s(i,n) < T(s)(o,n).$$

The output coordinate o is arbitrary. The input coordinate i need only be visible in the finite prefix $i \le n$. This matches the finite maximum built into the reflection operation and is strong enough to follow explicit child coordinates through the universal W-tree.

The repository already provides:

- `spliceTransformers`, which combines a countable family of transformers by selecting one at each output coordinate;
- `runTransformer`, which iterates a transformer along a well-founded W-tree;
- `liftTransformer`, which runs a transformer along the universal index tree;
- proofs that autonomous lifting preserves progressivity and strictly exceeds its input transformer at every visible coordinate.

Those facts are the reusable foundation for the new hierarchy.

4 A generic stage for arbitrary transformer lifts

The existing operator stage hard-codes `liftTransformer`. To iterate lifting itself, one first abstracts over the local lift.
Definition 4.1 (Transformer lift)**.** A transformer lift is an endomap on transformers:

$$\mathcal{L} := \mathcal{T} \to \mathcal{T}.$$

For $L \in \mathcal{L}$, base transformer B, and well-founded schedule a, define

$$\mathsf{Stage}(L, B, \mathsf{leaf}) := L(B),$$

and

$$\mathsf{Stage}(L, B, \mathsf{node}(f)) := L\big(\mathrm{splice}(i \mapsto \mathsf{Stage}(L, B, f(i)))\big)\,.$$

The corresponding Lean definition is only six lines:

Listing 1: Schematic ASCII rendering of `stageLift`.

```
abbrev TransformerLift := Transformer -> Transformer

def stageLift (lift : TransformerLift) (base : Transformer) :
    WTree Nat -> Transformer
  | .leaf   => lift base
  | .node f =>
      lift (spliceTransformers (fun i => stageLift lift base (f i)))
```

This is structural recursion on the W-tree. It is therefore executable and does not rely on an accessibility proof hidden behind an opaque definition. The old `operatorStage` is recovered by specializing L to `liftTransformer`.

Two predicates on lifts drive all later proofs.

Definition 4.2 (Preservation and strict inflation)**.** For $L \in \mathcal{L}$, let

$$\mathsf{Pres}(L) :\Longleftrightarrow \forall B,\ \mathsf{Prog}(B) \Rightarrow \mathsf{Prog}(L(B)),$$
$$\mathsf{Strict}(L) :\Longleftrightarrow \forall B,\ \mathsf{Prog}(B) \Rightarrow \forall s, i, o, n,\ i \le n \Rightarrow B(s)(i,n) < L(B)(s)(o,n).$$

The repository's autonomous lift has both properties. The new generic theorem is that a stage over a preserving lift remains progressive.

Theorem 4.3 (Stage progressivity)**.** *If L preserves progressive transformers and B is progressive, then*

$$\mathsf{Prog}(\mathsf{Stage}(L, B, a))$$

for every well-founded schedule a.

Proof architecture. Induct on a. The leaf case is exactly preservation. At a node, the induction hypothesis makes every child stage progressive. Splicing progressive child transformers is progressive, and one more application of preservation finishes the node case. This is formalized as `stageLift_progressive`. □

The strict counterpart is more delicate because there is no monotonicity assumption.

Theorem 4.4 (Base occurs strictly below every stage)**.** *Suppose L preserves progressivity and is strictly inflationary, and B is progressive. Then for every schedule a, seed s, coordinates i, o, and input n,*

$$i \le n \quad \Longrightarrow \quad B(s)(i,n) < \mathsf{Stage}(L, B, a)(s)(o,n).$$

Explicit-occurrence proof. At a leaf, strict inflation is exactly the desired statement. At a node $\mathsf{node}(f)$, follow child zero. By induction,

$$B(s)(i,n) < \mathsf{Stage}(L, B, f(0))(s)(0,n).$$

The right side is definitionally coordinate zero of the spliced child transformer. Since the splice is progressive, strict inflation of L places that coordinate below the parent. Transitivity completes the proof. No comparison between different oracle families is used. □

In Lean this theorem is `base_lt_stageLift`. It is the principal engine behind the diagonal hierarchy.

5 The diagonal hyperreflection hierarchy

Fix a schedule S. Define a hierarchy of lifts D_k^S by

$$\mathsf{D}_0^S(B) := \mathrm{liftTransformer}(B),$$
$$\mathsf{D}_{k+1}^S(B) := \mathsf{Stage}(\mathsf{D}_k^S, \mathsf{D}_k^S(B), S).$$

The Lean implementation is again direct structural recursion, now on k:

Listing 2: The diagonal hierarchy.

```
def diagonalLift (schedule : WTree Nat) : Nat -> TransformerLift
  | 0, base => liftTransformer base
  | k + 1, base =>
      stageLift (diagonalLift schedule k)
        (diagonalLift schedule k base) schedule
```

The successor clause contains the preceding rank in two roles:

1. *extensionally*, as the local lift at every node of the schedule;
2. *intensionally*, as the first amplification of the base transformer before the schedule traversal begins.

This is why the construction is called diagonal hyperreflection rather than plain finite iteration. It is not simply $L^k(B)$ for a fixed endomap L; the operator used at stage $k+1$ is itself the entire rank-k hierarchy.

5.1 Formal rank theorems

The generic facts imply two simultaneous inductions.

Theorem 5.1 (Every rank preserves progressivity)**.** *For every schedule S and $k \in \mathbb{N}$,*

$$\mathsf{Pres}(\mathsf{D}_k^S).$$

Theorem 5.2 (Every rank is strictly inflationary)**.** *For every schedule S and $k \in \mathbb{N}$,*

$$\mathsf{Strict}(\mathsf{D}_k^S).$$

At rank zero these are the existing theorems for `liftTransformer`. At a successor rank, progressivity follows from stage progressivity. Strict inflation first inserts the base below $\mathsf{D}_k^S(B)$ and then inserts $\mathsf{D}_k^S(B)$ below the stage built over it. The proof again uses explicit occurrences and transitivity.

The more informative comparison is rank-to-rank.

Theorem 5.3 (Strict rank growth)**.** *Let B be progressive. For every S, k, s, i, o, n with $i \le n$,*

$$\mathsf{D}_k^S(B)(s)(i,n) < \mathsf{D}_{k+1}^S(B)(s)(o,n).$$

This is the theorem `diagonalLift_rank_lt_succ`. It certifies an actual pointwise strict hierarchy, not merely a family of syntactically different definitions. In the final contender, k ranges through all naturals below a number already larger than the preceding contender's family value.

6 Dynamic-kind reflection over a countable family

Contender 13's reflector accepts a unary oracle $f : \mathbb{N} \to \mathbb{N}$. Operator-level amplification requires a countable family. The repository already has an injective packing convention `packFamily` that presents a family $s(i, m)$ through one unary input. The new reflector is

$$R_{\mathsf{Fd}}(s, n) := n + 1 + \max\left(\max_{i \le n} s(i, n), \max_{t \in \mathrm{CTm}_{\le n}} \mathrm{val}_{\mathsf{Fd}}(\mathrm{pack}(s), n, t)\right).$$

The actual Lean definition uses the repository's `prefixMax`, `maximum`, and `maxN` functions:

Listing 3: Dynamic-kind reflection with a family oracle.

```
def reflectFdFamily (fs : Family) (n : Nat) : Nat :=
  n + 1 + maxN (prefixMax (fun i => fs i n) n)
    (maximum (Fd.fvalue (packFamily fs) n) (Fd.ctmsUpTo n))

def fdFamilyTransformer : Transformer :=
  fun fs _ n => reflectFdFamily fs n
```

The visible prefix maximum is not redundant. Although every coordinate can in principle be reached through the packed unary oracle, an object-language enumeration theorem would require exhibiting a particular accessor program. The prefix maximum gives an immediate, syntax-independent inequality

$$i \le n \Longrightarrow s(i, n) < R_{\mathsf{Fd}}(s, n),$$

which is exactly the `Progressive` contract needed by the generic hierarchy.

The second maximum gives the language domination theorem:

$$t \in \mathrm{CTm}_{\le n} \Longrightarrow \mathrm{val}_{\mathsf{Fd}}(\mathrm{pack}(s), n, t) < R_{\mathsf{Fd}}(s, n).$$

Strong normalization of the dynamic-kind calculus makes `fvalue` total; the new layer does not add any new object-language reduction rule or consistency assumption.

7 Definition of Contender 14

7.1 Previous construction as a family

As required by the contest's non-laziness rule, the new definition does not mention the closed constant `contender_13`. It reuses the preceding *construction*:

$$\mathrm{family}_{13}(n) := F_{13}(F_{13}(n)).$$

Consequently, by definitional equality,

$$\mathrm{family}_{13}(42) = \mathrm{contender}_{13},$$

but the constant itself is absent from the transitive closure of the new definition. This is checked mechanically rather than trusted as a textual convention.

7.2 Input-dependent schedule and rank

The schedule is

$$S_{14}(n) := \mathrm{realize}\Big(\mathrm{hyper}_{\mathrm{family}_{13}(n)+42}(\mathrm{encode}(\mathrm{tree}_{12}))\Big).$$

The repository's `Autonomous.OTree` is transparent syntax for countably branching well-founded trees. Its operations mimic ordinal addition, multiplication, exponentiation, derivatives, common critical closures, and higher derivatives. The implementation deliberately does not quotient this syntax by ordinal equality. Therefore the article makes no theorem that $S_{14}(n)$ denotes a particular named ordinal. What is formal is that it is a well-founded executable schedule built by structural recursion, and that every node supplies explicit child occurrences used by the inequality proofs.

The rank is

$$r_{14}(n) := \mathrm{family}_{13}(n) + 1.$$

Both schedule complexity and diagonal rank therefore grow with the previous family. A fixed "42 levels" construction would be much weaker as a design: it would add a fixed meta-level above an already enormous language. Here the number of strict rank transitions is itself beyond the preceding family value at the same input.

7.3 Final family and number

The central definitions are:

Listing 4: Contender 14, schematic ASCII rendering.

```
def family13 (n : Nat) : Nat := F13 (F13 n)

def schedule14 (n : Nat) : WTree Nat :=
  Autonomous.realize
    (Autonomous.hyper (family13 n + 42)
      (Autonomous.encode tree12))

def rank14 (n : Nat) : Nat := family13 n + 1

def F14 (n : Nat) : Nat :=
  diagonalLift (schedule14 n) (rank14 n) fdFamilyTransformer
    (fun _ => family13) n n

def contender_14 : Nat := F14 (F14 42)
```

Every recursion visible here or in its dependencies is structural: finite list enumeration, strongly normalizing term evaluation, W-tree recursion, OTree recursion, and natural-number recursion. The definition is a closed `Nat` and is computable in the ordinary intensional sense. As with every serious BusyLean contender, attempting to normalize the final numeral is physically infeasible and should never be done.

8 Formal comparison and domination results

8.1 The previous family is below the new family

The final transformer is progressive, because the base transformer is progressive and every diagonal rank preserves progressivity. Instantiating progressivity with coordinate zero of the constant seed

family gives:
Theorem 8.1 (Pointwise family domination)**.** *For every $n \in \mathbb{N}$,*

$$\mathrm{family}_{13}(n) < F_{14}(n).$$

This is `family13_lt_F14`. It is the most direct formal statement that the new construction amplifies the previous one at every input, not only at the distinguished input 42.

8.2 Raw dynamic-kind reflection is below the hyperreflection layer

Strict inflation of every diagonal rank yields:
Theorem 8.2 (Raw reflection margin)**.** *For every n,*

$$R_{\mathsf{Fd}}(\lambda i.\,\mathrm{family}_{13}, n) < F_{14}(n).$$

Thus the diagonal layer is formally above an entire dynamic-kind reflection step applied to the previous family. This rules out the interpretation that the new result is only the old family plus an administrative wrapper.

8.3 Every enumerated dynamic-kind program is below F_{14}

Combining program domination by R_{Fd} with the raw-reflection margin gives:
Theorem 8.3 (Program domination)**.** *For every n and every dynamic-kind Church term t in the depth-n enumeration,*

$$\mathrm{fvalue}_{\mathsf{Fd}}(\mathrm{packFamily}(\lambda i.\mathrm{family}_{13}), n, t) < F_{14}(n).$$

The formal name is `fvalue_lt_F14`. The theorem applies to the same soundly type-checked and strongly normalizing language used by Contender 13, but with access to the whole previous family through the packed oracle.

8.4 The required contender comparison

The reflector contains $n+1$, so $F_{13}(n) > n$; applying this twice shows $\mathrm{family}_{13}(n) > n$. Pointwise family domination then gives $F_{14}(n) > n$. At $n = 42$,

$$\mathrm{contender}_{13} = \mathrm{family}_{13}(42) < F_{14}(42) < F_{14}(F_{14}(42)) = \mathrm{contender}_{14}.$$

Lean checks exactly this transitive chain:
Theorem 8.4 (Required contest theorem)**.**

$$\mathit{contender_13} < \mathit{contender_14}.$$

The theorem is `contender_13_lt_contender_14`. Combining it with the accepted comparison for Contender 13 gives `contender_12_lt_contender_14`, and transitivity extends to the entire preceding ladder.

9 Why the gain is structurally significant

The phrase "significantly exceeds" should be supported by theorem structure, not rhetoric. Four features distinguish the proposal from a trivial successor construction.

9.1 A previous-family-sized strict rank chain

At input n, the chosen rank is $\mathrm{family}_{13}(n) + 1$. The theorem `fdDiagonal_rank_lt_succ` says that every adjacent pair of ranks is strictly ordered at every visible coordinate. Hence the final rank is reached only after a formally certified sequence of strict transformer amplifications indexed by all $k < \mathrm{family}_{13}(n) + 1$.

The development does not materialize this chain as a list—doing so would be gratuitously expensive and unnecessary. The universally quantified rank theorem is stronger: any desired adjacent comparison is obtained by instantiation.

9.2 Two uses of the preceding rank at each successor

The successor equation

$$\mathsf{D}^S_{k+1}(B) = \mathsf{Stage}(\mathsf{D}^S_k, \mathsf{D}^S_k(B), S)$$

uses rank k as both operator and base amplification. This is qualitatively different from repeating a single fixed function. Each level changes the language of lifting available at every node of the schedule.

9.3 Countable-family access at the strongest language level

Contender 13's dynamic-kind programs receive a unary oracle. The new reflector packs a countable family into that oracle, while also retaining an explicit prefix maximum for unconditional progressivity. Thus each local reflection node can dynamically access values derived from different children and coordinates, rather than receiving only one frozen predecessor value.

9.4 Schedule growth is also previous-family indexed

The schedule is not fixed at `tree12`. It applies a number of higher-derivative constructors equal to $\mathrm{family}_{13}(n)+42$ above the encoding of that existing operator schedule. This second diagonalization is intentionally separated from the rank diagonalization: even a skeptical reader who assigns no ordinal semantics to `Autonomous.hyper` still has the rank theorem; a reader interested in ordinal-style schedules can study the additional tree growth independently.

10 Computability, constructivity, and axiom profile

10.1 Computable in principle

There are two notions that must not be conflated:

Computability. A finite program determines the unique natural number and could, given unbounded time and storage, evaluate it.

Feasible evaluation. The value can be materialized by current or physically possible hardware.

The first holds; the second emphatically does not. BusyLean is a largest-number contest, so infeasibility is expected. The relevant formal facts are that the definition is closed, total, structurally recursive, and free of semantic oracles such as a halting predicate.

Dynamic-kind program evaluation is total because the repository proves strong normalization and supplies a total normalizer over a term paired with its normalization certificate. Proof terms are

computationally irrelevant to the decoded natural number. The new meta-level hierarchy adds only primitive structural recursion over natural numbers and well-founded inductive trees.

10.2 Axiom audit

Lean reports:

Declaration	Reported axioms
`contender_14`	`[propext, Quot.sound]`
`contender_13_lt_contender_14`	`[propext, Quot.sound]`
`family13_lt_F14`	`[propext, Quot.sound]`
`fvalue_lt_F14`	`[propext, Quot.sound]`
`diagonalLift_rank_lt_succ`	`[propext]`

These are exactly within the repository's allowed set. No new axiom is introduced by diagonal hyperreflection.

10.3 Closure audit

A custom Lean command traverses every constant occurring in the type and value of `contender_14`, recursively following definitions and theorems. The validated audit states that the closure avoids all nine constants

`contender_5`, . . . , `contender_13`.

The definition legitimately depends on prior *families*, interpreters, normalizers, and schedules. What it does not do is insert a previously computed winner as an opaque seed.

10.4 Forbidden-construct scan

The validation workflow rejects the following constructs in the four new definition/proof modules:

`noncomputable`, `partial`, `unsafe`, `opaque`, `implemented_by`, `extern`, `native_decide`, `sorryAx`, and `Classical.choice`.

This textual guard is not a substitute for the kernel or axiom audit, but it catches common ways a superficially total definition can evade the intended contest rules.

11 Machine validation

The implementation was checked from a clean Ubuntu 24.04 GitHub Actions runner with Lean 4.34.0 and Lake 5.0.0. Two passes were used.

11.1 Full clean dependency pass

The first pass built all definition dependencies through Contender 14 and then all proof dependencies through Comparison 14:

Phase	Measured time	Budget
Definitions, including all prior definition dependencies	7.27 s	15 s
Proofs, including all prior proof dependencies	3.24 s	60 s
Axiom/smoke audit	passed	—

This is stronger than the repository's incremental rule because the measured phases included substantial accepted history.

11.2 Budget-faithful cached pass

The final workflow first builds `BusyLean.Comparison13` as the already accepted predecessor, then times only the new definition and proof layers. The validated results are:

Phase	Measured time	Budget
New definition layer	0.54 s	15 s
New proof layer	0.65 s	60 s
Source guard	passed	—
Axiom and closure audit	passed	—

Timing is inherently machine dependent, but the margin is large enough that ordinary runner variation is immaterial.

The validated branch is `research/contender14-diagonal-hyperreflection`; the final validation commit is `d408935e8a24da36709886bb08af3439765d05bf`. The default branch was not modified.

12 Proof-theoretic interpretation: what is and is not claimed

Transfinite iteration, Veblen progressions, and reflection hierarchies are standard tools for organizing proof-theoretic strength. Hyperations provide canonical transfinite iterations of normal ordinal functions and refine Veblen progressions [4]. Reflection ranks can coincide with proof-theoretic ordinals in appropriate settings [5]. These themes motivate the terminology and schedule design.

However, the BusyLean construction differs from those mathematical frameworks in important ways:

1. A reflected program can branch on oracle values, so the reflection transformer is not generally monotone in its oracle.
2. `Autonomous.OTree` is raw syntax for trees, not a quotient with a proved ordinal interpretation.
3. The diagonal hierarchy acts on concrete bounding transformers, not on theories ordered by consistency or on normal ordinal functions.
4. No theorem in the present development identifies `contender_14` with a fast-growing hierarchy value at a named ordinal.

Accordingly, any ordinal description is heuristic. One may reasonably view the construction as adding a previous-family-sized hierarchy of meta-reflection operators above a dynamic-kind language, with a second previous-family-sized expansion of the scheduling syntax. It would be unjustified to print an exact ordinal label without first formalizing a semantic interpretation, a well-ordering theorem for the schedule notation, and a correspondence theorem between that interpretation and the numerical reflector.

This conservative stance is a strength, not a weakness. The numerical comparison is fully formal and does not depend on speculative ordinal calibration. The heuristic layer is isolated as a research program.

13 Threats to validity and adversarial review

13.1 Could the definition hide the previous contender?

No. The transitive closure audit rejects every earlier contender constant. The equality $\text{family}_{13}(42) = \text{contender}_{13}$ follows because both unfold to the same construction, not because the new definition references the constant.

13.2 Does the comparison rely on oracle monotonicity?

No. The proof of `base_lt_stageLift` explicitly follows child zero into a splice and then applies strict inflation to that concrete coordinate. Every later inequality reduces to this occurrence argument.

13.3 Is `Autonomous.hyper` semantically overclaimed?

No. The code and article describe it as tree syntax inspired by ordinal operations. The contender theorem uses only structural recursion and progressivity. Even if two syntactically different trees represented the same ordinal under a future quotient, the Lean inequalities proved here would remain valid.

13.4 Does the huge rank cause nontermination of the definition checker?

No. Lean checks a structurally recursive equation once; it does not evaluate `rank14 42`. At runtime, evaluation terminates in principle because recursion decreases a natural-number argument. Kernel checking and numeral evaluation are distinct tasks.

13.5 Is the family reflector merely adding the seed by a maximum?

The prefix maximum is deliberately present to establish progressivity, but the final family is not just that maximum. It also bounds every dynamic-kind program and then enters a diagonal hierarchy whose each rank is a strict transformer amplification. The theorem `raw_reflection_lt_F14` isolates this extra margin.

13.6 Could the proof be vacuous because the term enumeration is empty?

No. The family domination and rank-growth theorems do not depend on the presence of any program in the enumeration. Program domination is an additional theorem. Moreover, the inherited enumerator contains ordinary variables, lambda terms, applications, type and kind abstractions/applications, packing constructs, and the oracle term at positive depth.

13.7 Are allowed axioms used computationally?

The reported axioms occur in semantic normalization infrastructure and extensional equality reasoning. They do not supply a noncomputable natural-number oracle. This matches the repository's existing accepted policy.

14 Integration plan

The archive is organized as an overlay rather than a replacement repository. The four new modules should be copied into `BusyLean/`:

1. `HyperReflection.lean`;
2. `HyperReflectionFacts.lean`;
3. `Contender14.lean`;
4. `Comparison14.lean`.

Then append imports for those modules to `BusyLean.lean`, copy `test/Contender14Audit.lean`, and optionally install the included validation workflow. The integration directory contains a script and patch snippets.

A repository maintainer may also update the global `scripts/check.py` constants from Contender 13 to Contender 14 and merge the new axiom/closure lines into `test/Audit.lean`. Those edits are intentionally supplied as integration artifacts rather than silently applied to the default branch.

The recommended commands are:

Listing 5: Reproduction commands after applying the overlay.

```
lake build BusyLean.Comparison13
/usr/bin/time -f '%e' lake build BusyLean.Contender14
/usr/bin/time -f '%e' lake build BusyLean.Comparison14
lake env lean test/Contender14Audit.lean
```

Never run `#eval BusyLean.contender_14`.

15 Further research questions

The present result opens several concrete directions.

15.1 Semantic calibration of diagonal hyperreflection

Construct a monotone surrogate semantics that records only explicit occurrence dependencies, then assign ordinal diagrams to `stageLift` and `diagonalLift`. The central challenge is to account for branching oracle programs without falsely treating reflection as pointwise monotone.

15.2 Transfinite rather than natural rank syntax

Replace the natural rank k with a W-tree-indexed or OTree-indexed recursion on transformer lifts. A serious version should include a generic progressivity/strictness theorem and a well-founded recursor whose computational content remains transparent. This could yield a genuine autonomous meta-meta-reflection closure rather than a previous-family-sized finite rank.

15.3 Internalizing family access in the object language

At present the countable family is encoded through a unary pairing function. Extend dynamic-kind System Fω with a typed family oracle or a tree-valued oracle, prove strong normalization, and show that family selection is represented directly. This may remove packing overhead and expose richer dynamic scheduling to reflected programs.

15.4 A predicative ladder of dynamic kinds

Contender 13 has one distinguished `dyn` kind containing all dyn-free operators. Introduce `dyn 0`, `dyn 1`, ..., where each level may package operators over lower levels but not itself. Prove normalization uniformly and diagonalize over the level index. This would be a language-level advance rather than solely an amplifier advance.

15.5 Large elimination with realizability

The repository's roadmap points toward dependently typed languages with large elimination. A realizability model that permits natural-number data to select types or kinds at runtime could produce a more qualitative jump than further fixed-language reflection. The main deliverable would need to be a kernel-checked total interpreter and sound checker, not only a proposed syntax.

15.6 Compression and kernel performance

The current definitions check quickly, but future contenders may hit elaboration or reduction bottlenecks before mathematical limits. Study defunctionalized transformer codes, proof irrelevance boundaries, and generated recursion lemmas that reduce kernel term size without weakening transparency.

15.7 Formal contest dominance beyond the immediate predecessor

Build a generic theorem that, given a chain of family equalities and pointwise inequalities, automatically derives domination over every earlier contender. This would reduce repetitive comparison modules and make audit obligations more declarative.

15.8 Independent mechanization

Port the generic diagonal hierarchy to another proof assistant or to a small verified kernel model. Agreement would be valuable evidence that no Lean-specific elaboration feature is carrying hidden mathematical content.

16 Conclusion

Contender 14 combines the repository's strongest language with a new, input-indexed hierarchy of reflection operators. Its central recursion is concise, but the comparison is not superficial: the development proves progressivity, strict inflation, adjacent-rank growth, raw-reflection domination, program domination, pointwise domination of the entire previous family, and the required closed-number inequality. The proof deliberately avoids oracle monotonicity and instead tracks explicit occurrences through spliced child transformers.

The result is constructive, computable in principle, within the permitted axiom set, below the time budgets by a wide margin, and free of direct dependencies on earlier contender constants. It is therefore suitable as a serious candidate for integration into BusyLean. Its most important conceptual contribution is reusable: *reflection can be lifted not only from functions to families and from families to transformers, but into a strictly growing hierarchy of the lifting operations themselves.*

A Formal theorem inventory

Lean declaration	Mathematical role
`stageLift_progressive`	A schedule built with a progressivity-preserving lift is progressive.
`base_lt_stageLift`	A strict preserving lift places the base below every scheduled stage.
`diagonalLift_preservesProgressive`	Every diagonal rank preserves progressivity.
`diagonalLift_strictlyInflationary`	Every diagonal rank strictly exceeds its original base.
`diagonalLift_rank_lt_succ`	Consecutive diagonal ranks are pointwise strictly increasing.
`oracle_lt_reflectFdFamily`	Every visible oracle-family coordinate is below family reflection.
`fvalue_lt_reflectFdFamily`	Every enumerated dynamic-kind program is below family reflection.
`fdFamilyTransformer_progressive`	Dynamic-kind family reflection satisfies the generic contract.
`fdTransformer_lt_diagonal`	Every diagonal rank exceeds raw family reflection.
`fdDiagonal_rank_lt_succ`	The concrete dynamic-kind hierarchy has strict adjacent ranks.
`family13_lt_F14`	The previous construction is dominated at every input.
`raw_reflection_lt_F14`	A whole dynamic-kind reflection layer lies below F_{14}.
`fvalue_lt_F14`	All current-budget dynamic-kind programs lie below F_{14}.
`input_lt_F14`	F_{14} grows past its numeric input.
`contender_13_lt_contender_14`	Required contest comparison.

B Artifact map

`article/` LaTeX source and compiled PDF.

`overlay/BusyLean/` Four Lean modules implementing the construction and proofs.

`overlay/test/` Axiom, closure, theorem-signature, and small execution audit.

`overlay/.github/workflows/` Reproducible Lean 4.34.0 validation workflow.

`integration/` Application script, import patch, check-script guidance, and maintainer notes.

`validation/` Machine logs, theorem summary, source scan, and branch metadata.

`SHA256SUMS` Integrity hashes for the packaged files.

References

[1] V. Reshetnikov, *BusyLean: name the biggest number, in Lean*, GitHub repository, 2026. https://github.com/VladimirReshetnikov/BusyLean

[2] J.-Y. Girard, *Interprétation fonctionnelle et èlimination des coupures de l'arithmétique d'ordre supérieur*, Thèse d'État, Université Paris VII, 1972.

[3] K. Donnelly and H. Xi, "A Formalization of Strong Normalization for Simply-Typed Lambda-Calculus and System F," *Electronic Notes in Theoretical Computer Science* 174(5), 109–125, 2007. https://doi.org/10.1016/j.entcs.2007.01.021

[4] D. Fernández-Duque and J. J. Joosten, "Hyperations, Veblen Progressions and Transfinite Iterations of Ordinal Functions," arXiv:1205.2036, 2012. https://arxiv.org/abs/1205.2036

[5] F. Pakhomov and J. Walsh, "Reflection Ranks and Ordinal Analysis," *Journal of Symbolic Logic* 86(4), 1350–1384, 2021. https://doi.org/10.1017/jsl.2020.9

[6] L. de Moura and S. Ullrich, "The Lean 4 Theorem Prover and Programming Language," in *Automated Deduction–CADE 28*, LNCS 12699, 625–635, 2021. https://doi.org/10.1007/978-3-030-79876-5_37